GigHub: Application Design and Development Report(MVP)

Course Code: CSEP-3210
Course Name: Application Design and Development (Project)
Submission Date: July 27, 2026

Submitted To:

Md. Abu Layek, PhD
Professor and Chairman,
Department of Computer Science & Engineering
Jagannath University, Dhaka

Submitted By:

Team Trityobit

Name	Student ID
Maisha Binte Monir	B210305016
MD. Ahsanul Hoque Abir	B210305040

Department of Computer Science and Engineering
Jagannath University, Dhaka, Bangladesh

Contents

Chapter 1

Introduction

1.1 Product Overview

GigHub is a campus-exclusive freelance marketplace for Jagannath University (JnU) students. Every student operates from a single account simultaneously as both buyer and seller; no role selection required. This is designed to provide students with easier access to freelance opportunities, enabling them to develop practical skills, gain real-world experience, and build valuable professional connections within the university community.

Beyond freelancing, GigHub serves as a bridge between current students and alumni, creating opportunities for collaboration, mentorship, networking, and project-based work. By fostering a supportive ecosystem where talent, services, and opportunities circulate within the JnU community, the platform aims to generate both economic and social benefits.

1.2 Problem Statement

- Lack of a trusted freelancing platform for students
- Difficulty connecting students with peers who need their skills
- Fragmented access to campus opportunities
- Risk of fraud and payment disputes
- Limited trust and accountability mechanisms
- Lack of structured peer-to-peer learning opportunities

1.3 Core Objectives

- Trusted campus-only network with verified student identity
- Gig marketplace + job board + peer tutoring in a single platform
- Escrow-backed transaction safety for paid services
- Real-time communication and review-based trust ratings
- Free peer tutoring/tuition exchange with no platform fee

1.4 Scope of the Project

- Secure authentication and student identity management
- Profile creation and management
- Gig marketplace with tiered packages
- Job posting and proposal flow
- Peer tutoring/tuition exchange (free, no platform fee)
- Order lifecycle with delivery, revision, and completion
- Escrow-backed payment and withdrawal
- Real-time messaging and push notifications
- Mutual review and rating system
- Unified student dashboard (buyer + seller views)
- Admin moderation panel and dispute handling

1.5 Key Differences from Generic Platforms

- Exclusively for Jagannath University, providing a trusted and closed campus ecosystem.
- Every student can act as both a buyer and a seller, eliminating the need to choose a fixed role.
- A unified backend powered by Supabase (PostgreSQL, Authentication, and Realtime Broadcast), removing the need for separate database, authentication, and WebSocket services.
- Cloudflare R2 stores all files, while Cloudinary is dedicated to image optimization and delivery, providing specialized storage and content delivery layers.
- Shared Zod schemas and TypeScript types through `@gig-hub/types` ensure consistent validation and type safety across both web and mobile applications.
- Supports free and volunteer tasks in addition to paid freelance gigs.
- Student-first, mobile-first user experience built with React Native and a lightweight onboarding process.
- Integration with SSLCommerz enables secure escrow-based payments tailored for the Bangladesh market.

Chapter 2

Requirements Specification

2.1 Functional Requirements

Here, **Notation: P0= Must have, P1= Should have**

FR-A Authentication & Identity: Email/password + Google OAuth, secure session tokens, server-side verification, and automatic profile creation.

FR-B Profile Management: View/edit profile, avatar upload, skills, public profile, verification badge.

FR-C Gig Marketplace: Create/edit gigs with package tiers, images, listings, and basic search/filters.

FR-D Job Board & Proposals: Post jobs, submit/accept proposals; accepted proposals create orders.

FR-E Orders & Delivery: Order lifecycle (pending → in-progress → delivered → completed/disputed), deliveries, revisions, approvals.

FR-F Payments & Escrow: Escrow on payment, release on approval, platform fee (5% up to BDT 500), payment gateway integration, withdrawals, transaction history.

FR-G Messaging & Notifications: Real-time 1:1 chat, order-linked channels, attachments, in-app notifications and push.

FR-H Reviews & Trust: Mutual reviews and ratings, text feedback, aggregated scores on profiles and listings.

FR-I Dashboard & Admin: Unified user dashboard, earnings and transactions, admin moderation and dispute queue.

FR-J Tuition Listings: Free-form tuition listings (title + description), request/accept flow opens chat, no payments involved.

2.1.1 Acceptance Criteria

- Core P0 flows (auth, order lifecycle, payments, chat, reviews) work end-to-end and are testable
- Financial actions are idempotent and auditable
- Public data exposure is restricted to allowed fields

2.2 Non-Functional Requirements

- **Performance:** p95 read latency<200ms, chat latency<1s, search<500ms
- **Scalability:** Design for horizontal scaling to support 5,000+ concurrent users
- **Availability:** Target 99.5% uptime with graceful degradation of non-core features
- **Security:** OWASP controls, token-based auth, input validation, TLS for all traffic
- **Usability:** Mobile-first, bilingual-friendly, aim for WCAG 2.1 AA
- **Maintainability:** Modular services, clear API versioning, replaceable components
- **Observability:** Structured logs, key metrics, and alerting for errors/latency spikes

2.3 User Requirements

2.3.1 Student User

Every registered JnU student has full dual-sided and tutoring access from a single account:

Capability	Description
Create Gigs	List paid services with package tiers
Browse and Order Gigs	Purchase services from peers
Post Jobs	Post task requests with budget and deadline
Submit Bids	Apply to peer job postings
Post Tutoring Sessions	Offer tutoring/lessons free of charge
Request Tutoring	Browse and request available tutors
Chat	1-to-1 messaging for inquiries and active orders
Manage Orders	Track orders as buyer and seller
Client Reviews	Rate and review completed transactions
Receive Payments	Withdraw earnings to bKash or bank
Make Payments	Pay via local payment gateway
Profile Management	Edit profiles, add credentials and contact info

2.3.2 Admin User

Capability	Description
User Management	View, verify, suspend, and ban accounts
Content Moderation	Review flags on gigs, jobs, tutoring posts, reviews
Dispute Resolution	Handle order disputes and process refunds
Platform Configuration	Set service fees, escrow timers, job categories
Analytics Visibility	View platform revenue, user growth, order stats

Chapter 3

System Design & Development

3.1 System Architecture

- **Presentation Layer:** Next.js Web App, React Native Mobile App
- **API Layer:** NestJS backend for routing, validation, and business logic
- **Application Layer:** Gig, Order, Escrow Payment, Messaging, Notification, and Review services
- **Auth & Real-Time Layer:** Supabase Auth, real-time subscriptions, file storage
- **Data Layer:** PostgreSQL relational database
- **Infrastructure Layer:** Dockerized VPS deployment with CI/CD support
- Supports secure transactions, scalability, and real-time communication

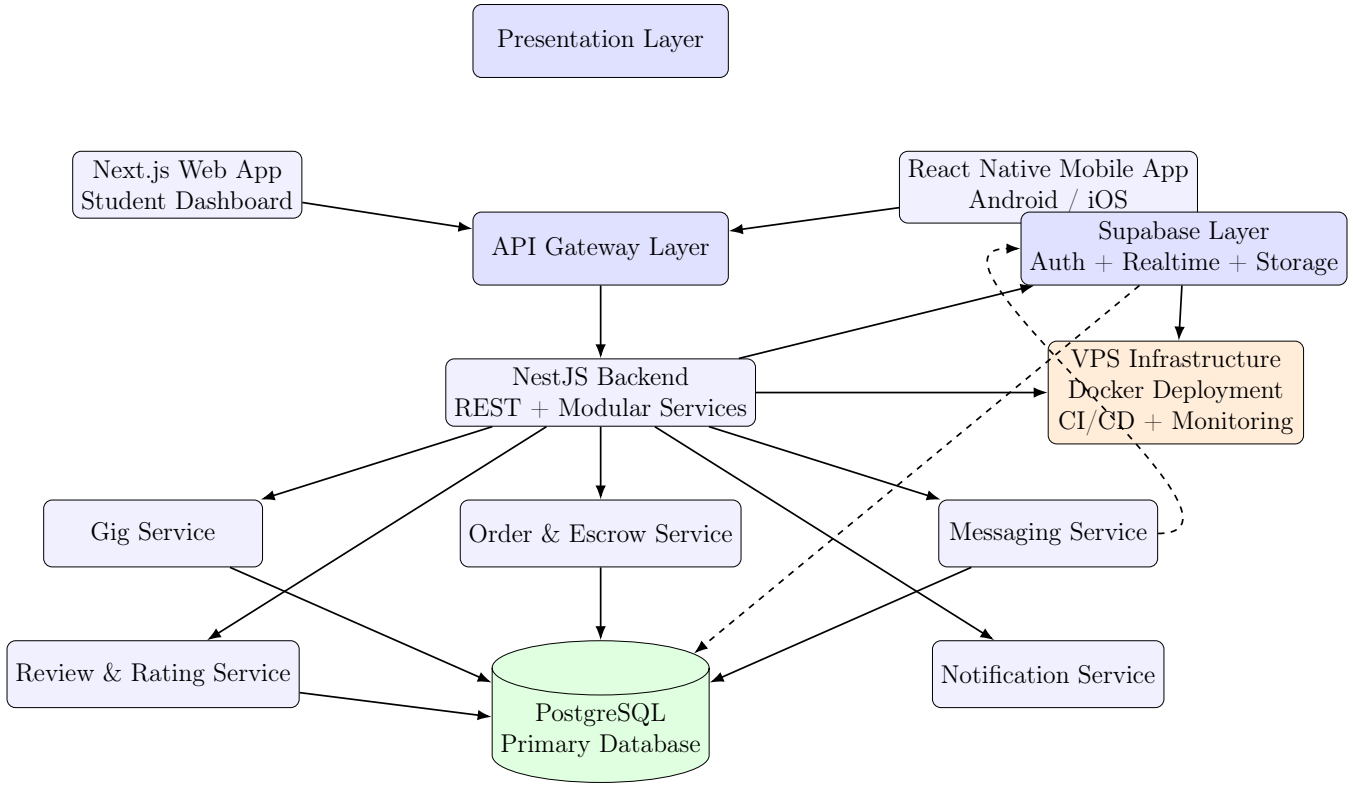


Figure 3.1: Detailed System Architecture of GigHub Platform

3.2 Deployment Architecture

The GigHub deployment architecture is designed to ensure scalability, reliability, and automated delivery using containerization and continuous integration practices. The system follows a Docker-based deployment model hosted on a VPS, with CI/CD pipelines handling automated build, testing, and deployment.

The entire application is containerized into multiple services, including the Next.js frontend, React Native web build (optional), NestJS backend API, and supporting services such as PostgreSQL and Supabase integration layer.

A Git-based CI/CD pipeline is used to automate deployment. Whenever code is pushed to the main branch, the pipeline triggers a build process, runs tests, creates Docker images, and pushes them to a container registry. The VPS then pulls the latest images and redeploys the services using Docker Compose.

The VPS acts as the central hosting environment where all containers are orchestrated. Nginx is used as a reverse proxy to route incoming traffic to appropriate services such as the frontend and backend APIs.

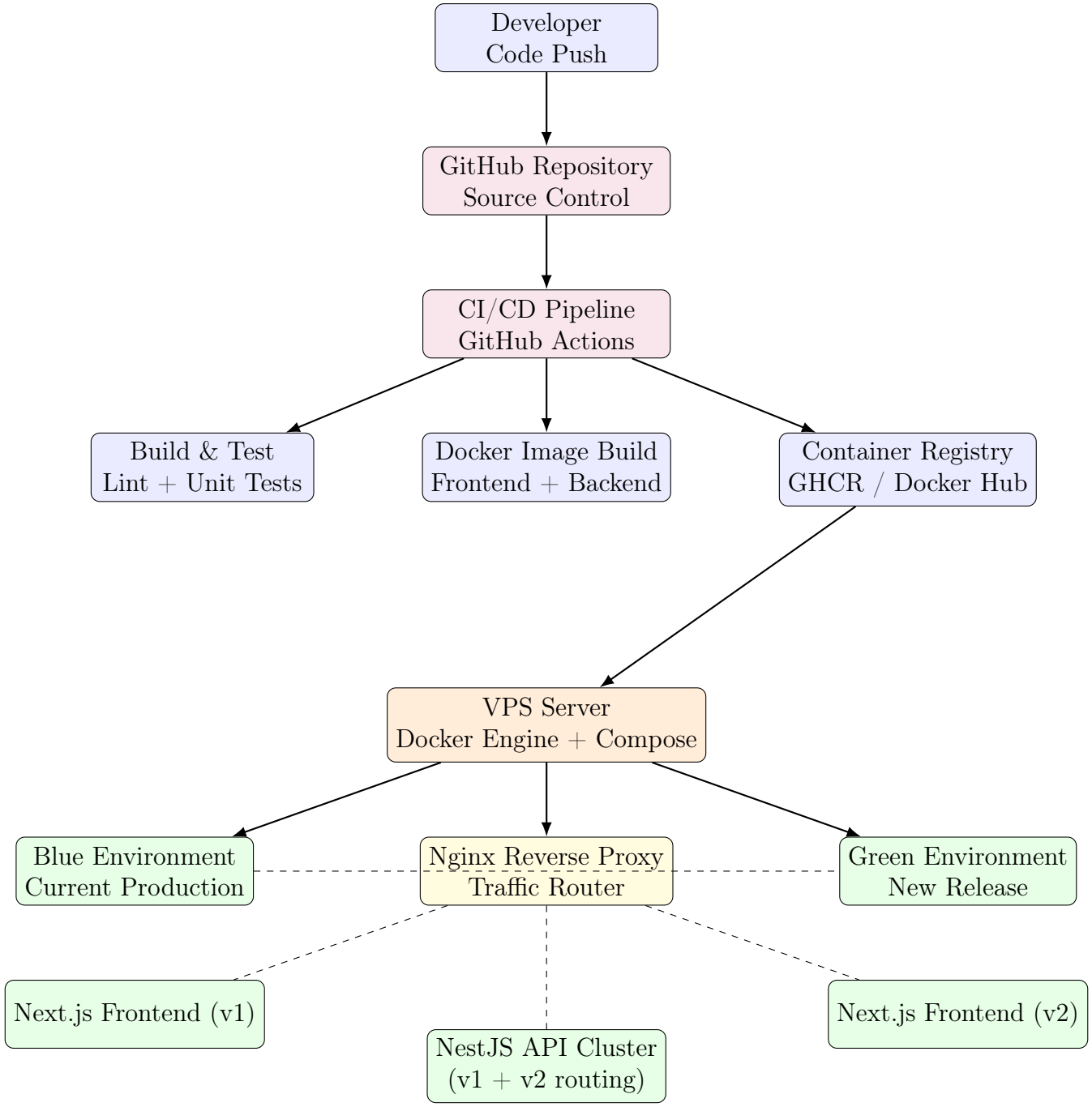


Figure 3.2: Zero-Downtime CI/CD Deployment Architecture for GigHub

3.3 Database Design

The GigHub system is designed using a relational database model following 3NF normalization principles. The database supports core functionalities such as user management, gig operations, order processing, escrow-based payment handling, messaging, and reviews.

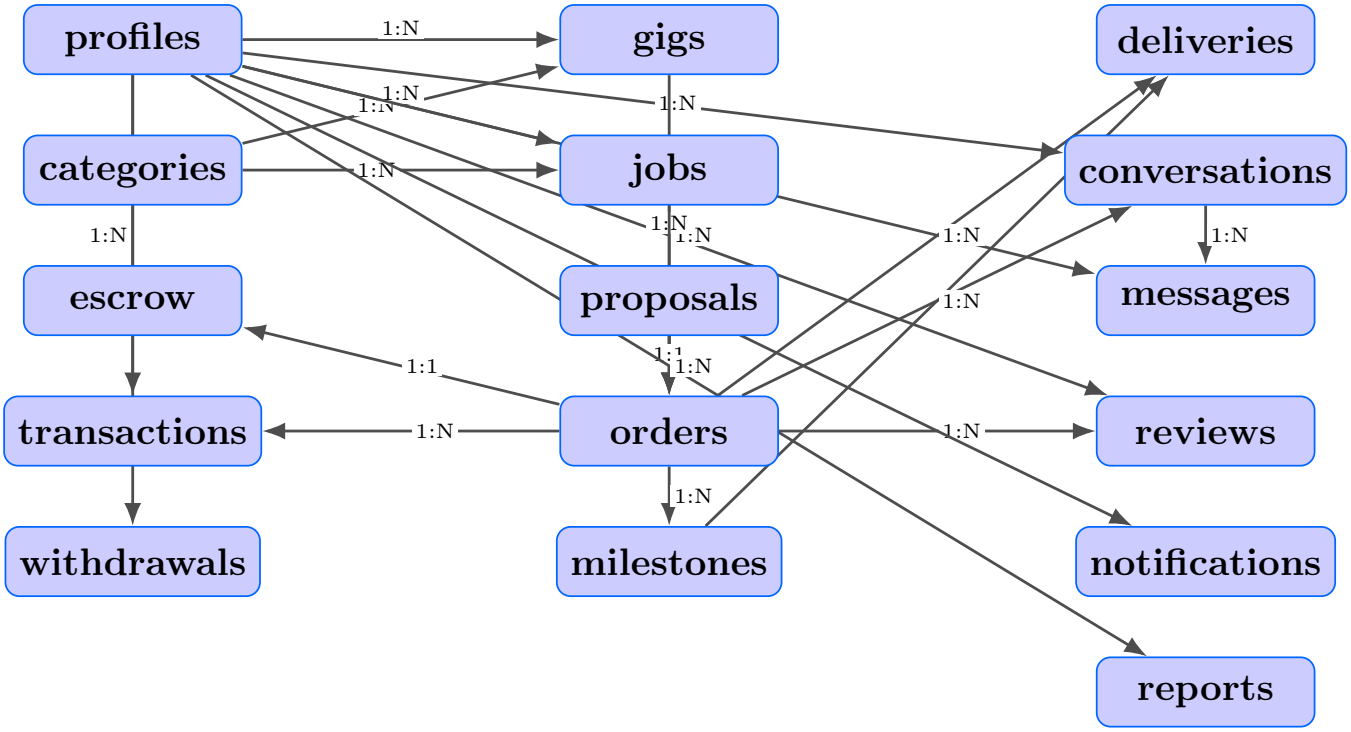


Figure 3.3: Entity relationship diagram

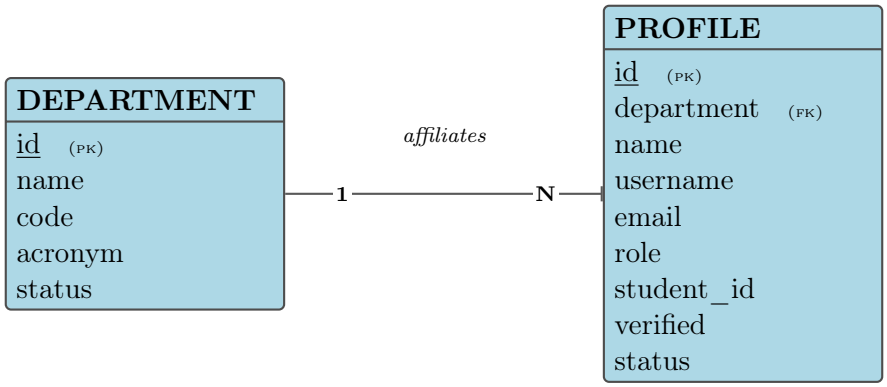


Figure 3.4: Module 1 – User & Identity Management: relationship between `DEPARTMENT` and `PROFILE`.

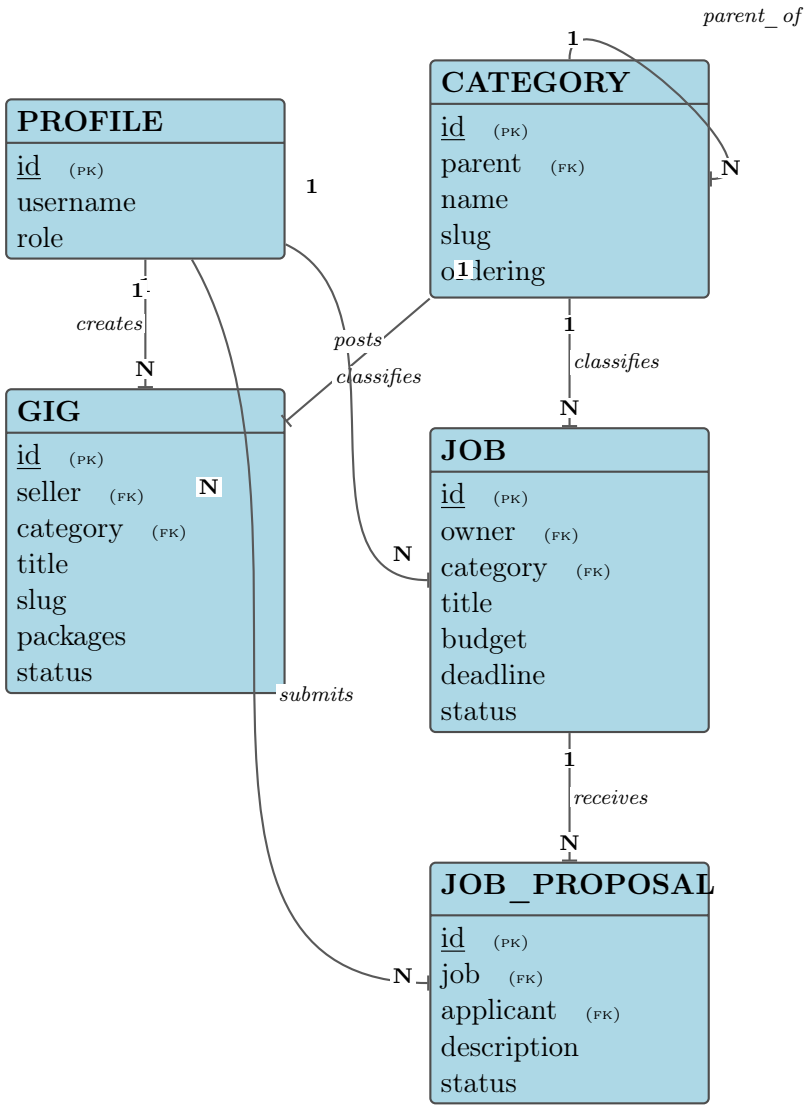


Figure 3.5: Module 2 – Marketplace (Catalog & Hiring): relationships between `CATEGORY`, `GIG`, `JOB`, `JOB_PROPOSAL`, and `PROFILE`.

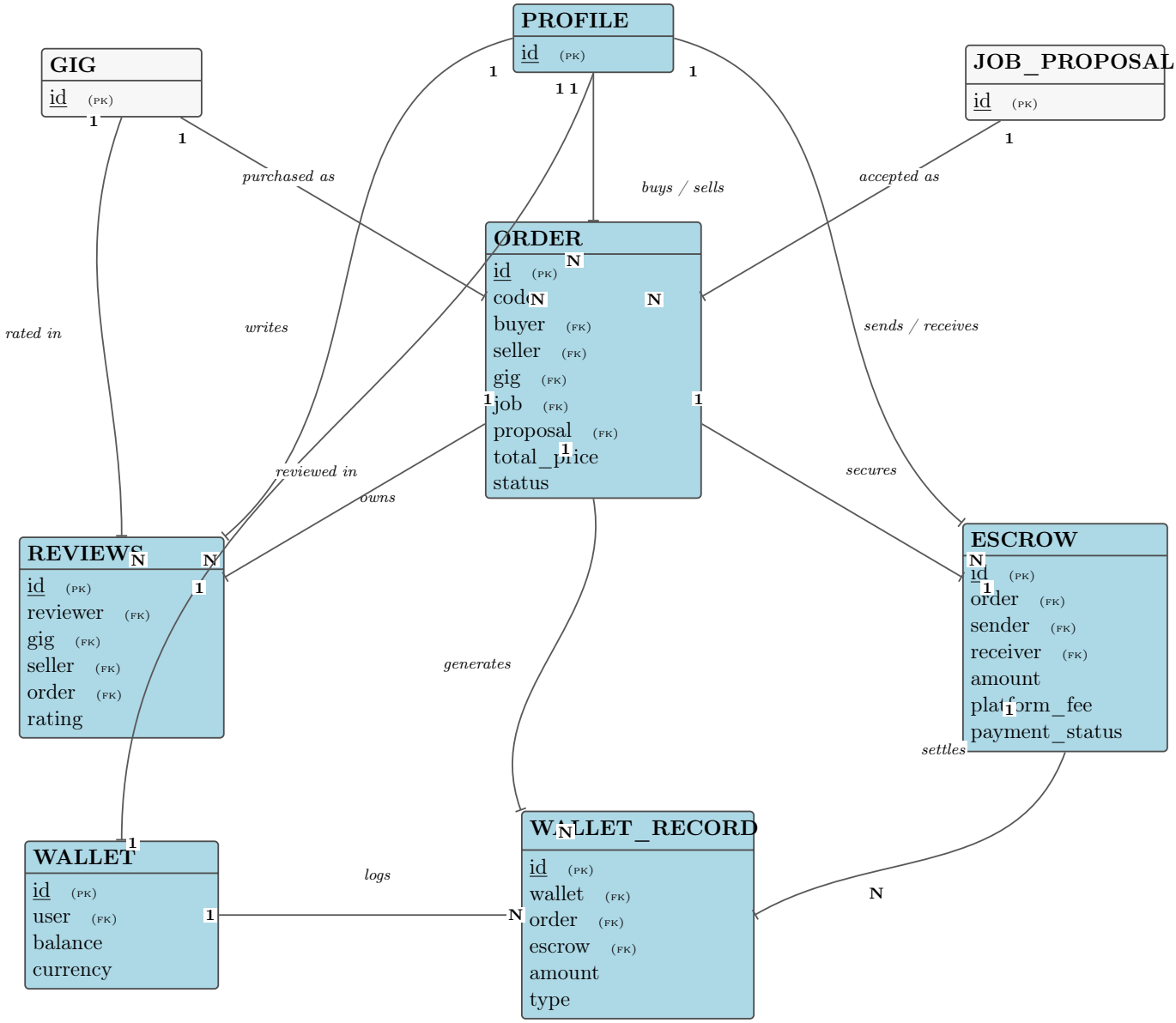


Figure 3.6: Module 3 – Orders, Escrow & Payments: the transactional core linking `ORDER`, `ESCROW`, `WALLET`, `WALLET_RECORD`, and `REVIEWS`. `PROFILE`, `GIG`, and `JOB_PROPOSAL` are shown as external references defined in Modules 1 and 2.

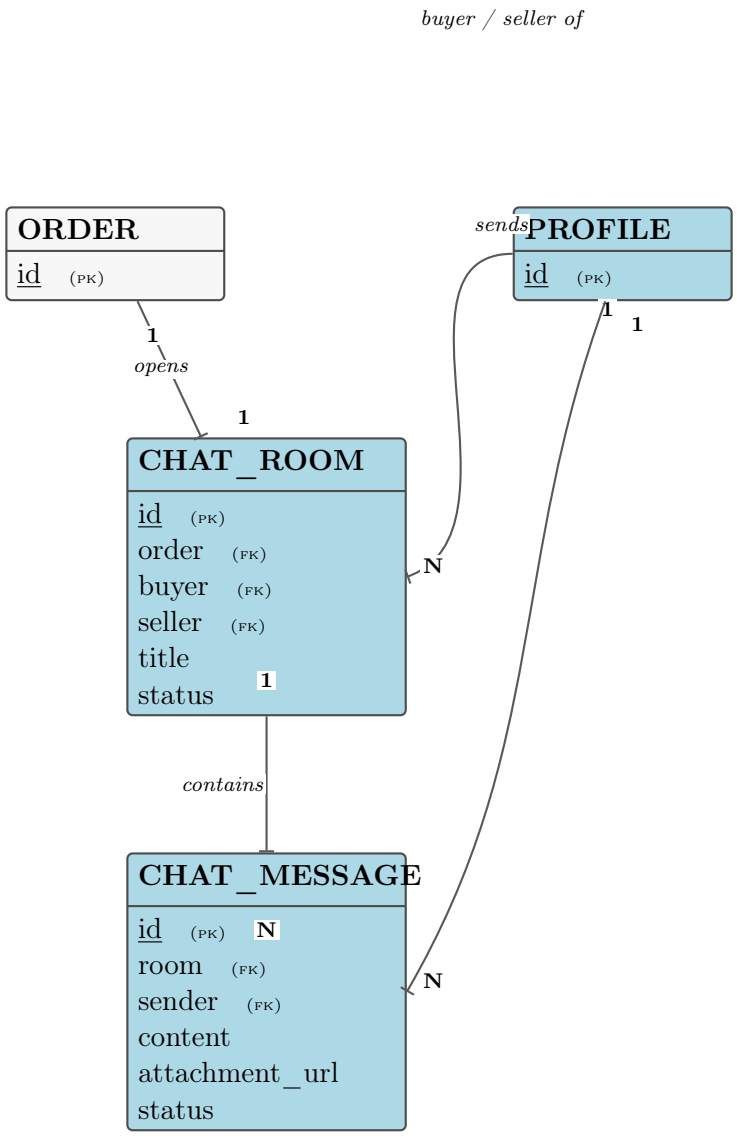


Figure 3.7: Module 4 – Communication: `CHAT_ROOM` and `CHAT_MESSAGE`, scoped to a single `ORDER`. `ORDER` and `PROFILE` are external references defined in Modules 1 and 3.

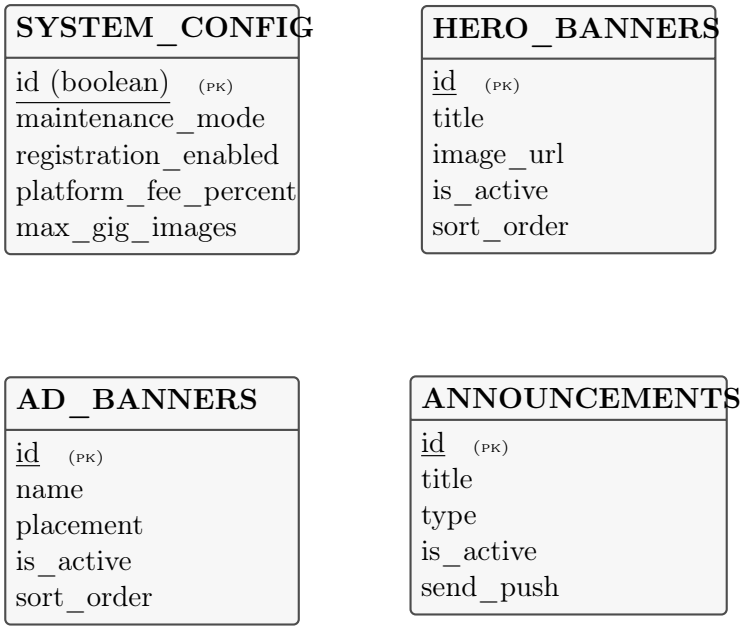


Figure 3.8: Module 5 – Platform Configuration & Content Management. These four tables carry no foreign keys to any other table in the schema and are therefore shown without connecting relationships.

Chapter 4

Implementation Details

The GigHub system was implemented as both a web-based and a mobile application to provide users with flexible access across different platforms. For demonstration and evaluation purposes, predefined user accounts were created to represent different system roles. The login credentials are provided below:

- **Administrator Account**
  - Email: `admin@gighub.com`
  - Password: `password123`
- **Student Account**
  - Email: `student@gighub.com`
  - Password: `password123`

The web application can be accessed through the following URL: `https://gighub.ahsanull.com`

4.1 Programming Languages

The GigHub platform is implemented using a combination of modern programming languages chosen to support both web and mobile development, as well as backend service orchestration.

- **JavaScript and TypeScript** are the primary languages used for the web application and backend services.
- The **Next.js frontend** is built on TypeScript to ensure type safety, maintainability, and improved developer experience.
- The **NestJS backend** is also developed using TypeScript, leveraging its strong typing system and modular architecture support.
- For the mobile application, **Dart** is used with the **React Native framework**, enabling fast UI rendering and cross-platform compatibility.
- A single codebase is used in React Native to run efficiently on both Android and iOS devices.
- **SQL** is used for database querying and schema definition in PostgreSQL, ensuring structured and relational data handling across the system.

4.2 Frameworks & Tools

GigHub utilizes a set of modern frameworks and development tools to ensure scalability, performance, and maintainability across the entire system.

- **Next.js** is used for building the web frontend, providing server-side rendering, routing, and optimized performance for the user interface.
- **React Native** is used for mobile application development, enabling a responsive and native-like experience across platforms.
- **NestJS** serves as the primary backend framework, offering a modular architecture, dependency injection, and support for scalable RESTful APIs.
- **Supabase** is integrated as an application layer over PostgreSQL, providing authentication, real-time capabilities, and storage services.
- **Docker** is used for containerization, ensuring consistent deployment environments across development and production.
- **GitHub Actions** (or similar CI/CD tools) automate the build, test, and deployment pipeline.
- **Nginx** is used as a reverse proxy to manage incoming traffic and route requests efficiently to backend and frontend services.
- Additional tools include **Git** for version control and VPS-based Linux servers for hosting and deployment.

4.3 Database System

The GigHub platform uses PostgreSQL as its primary relational database management system. PostgreSQL is chosen due to its reliability, scalability, and strong support for complex relational queries.

The database stores all core system entities including users, gigs, job posts, bids, transactions, messages, ratings, and administrative records. Relationships between entities are maintained using primary and foreign key constraints to ensure data integrity.

Supabase is used as an additional application layer over PostgreSQL, providing authentication, real-time data synchronization, and storage capabilities without replacing the core relational structure.

The database design follows normalization principles to reduce redundancy and improve data consistency. Indexing strategies are applied on frequently queried fields such as user IDs, gig categories, and transaction records to enhance query performance.

Overall, the database system ensures secure, structured, and efficient handling of all GigHub platform data.

Table 4.1: GigHub Technology Stack

Layer	**Technology**
API & Backend	Next.js API Routes (TypeScript, REST)
Web Frontend	Next.js (App Router, Server Components)
Mobile App	React Native (Android & iOS)
Database	Supabase (PostgreSQL + Realtime Broadcast)
Authentication	Supabase Auth (Email + Google OAuth)
File Storage	Cloudflare R2 for all files (documents, uploads, assets), served via Cloudflare CDN
Image CDN	Cloudinary for banners, gig images, and avatars (image optimization & transformations)
Shared Package	`@gig-hub/types` (Zod schemas + TypeScript types)
Real-time	Supabase Realtime Broadcast
Payments	SSLCommerz (escrow-based payment system)

4.3.1 Payment and Escrow Flow

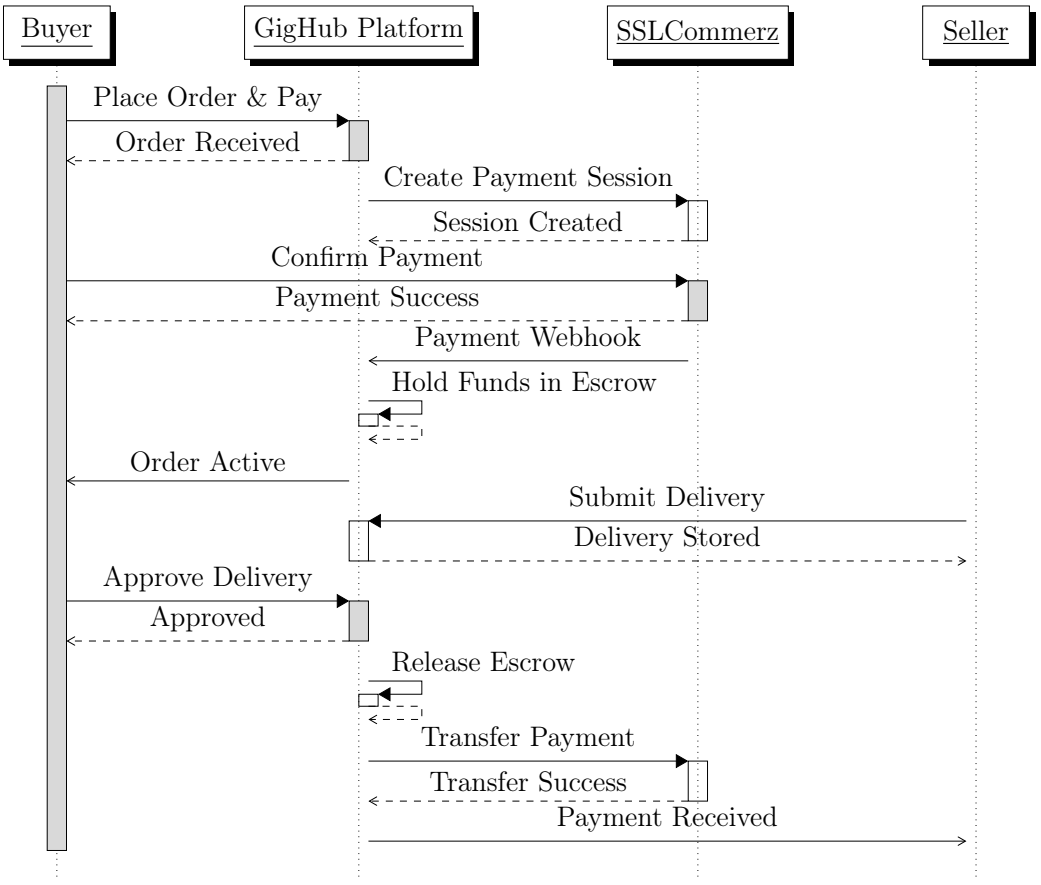


Figure 4.1: Payment and escrow workflow using SSLCommerz

4.3.2 Auth and Data Flow

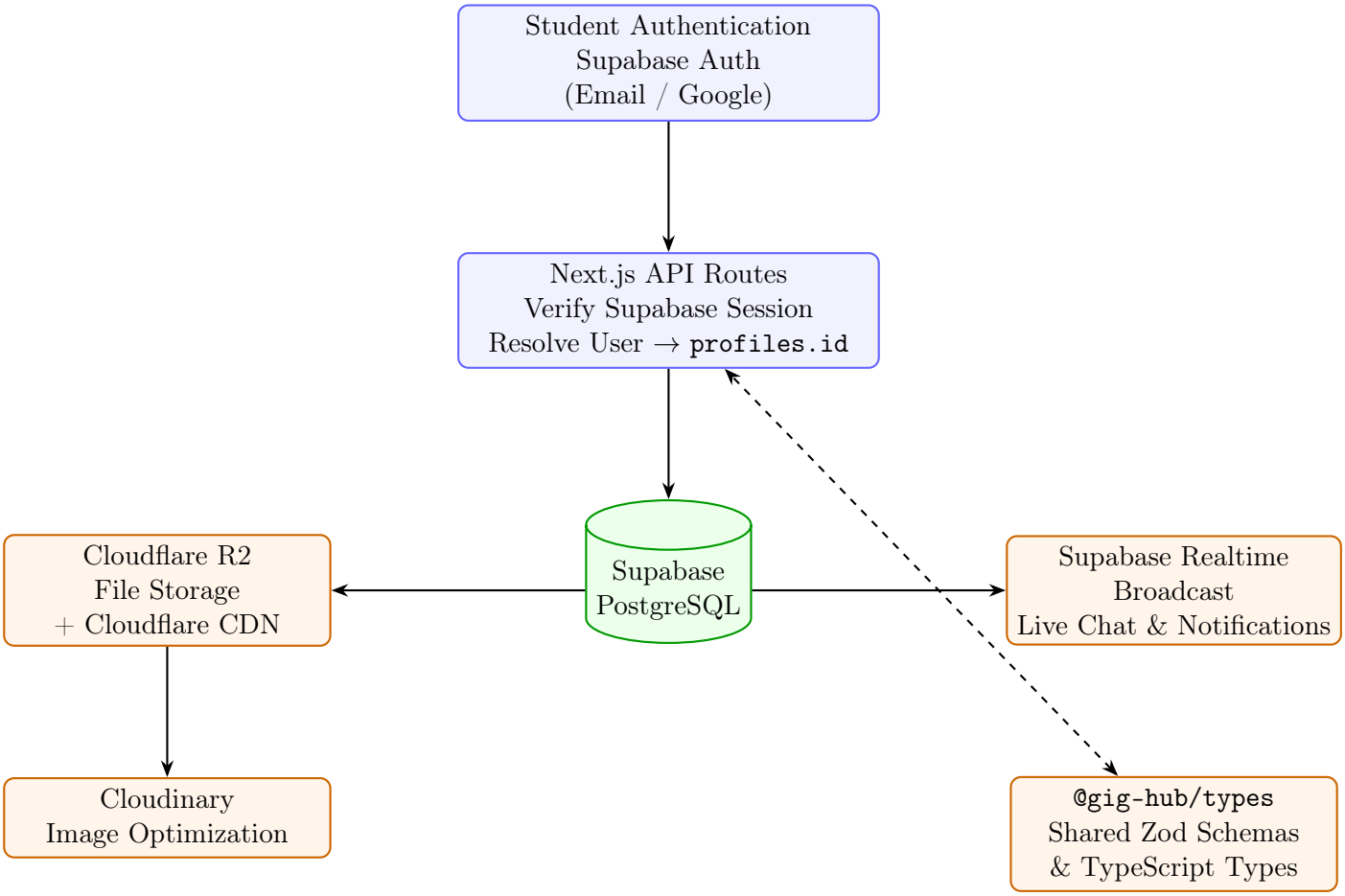


Figure 4.2: Backend architecture and service interaction in GigHub.

Chapter 5

Testing

Testing was conducted to ensure that GigHub functions correctly, securely, and reliably across all modules. The testing process consisted of Unit Testing, Integration Testing, and System Testing.

5.1 Unit Testing

Unit testing was performed by the developers on individual modules before integration. Each service was tested independently using dedicated test data to verify that its functionality met the specified requirements. The objective was to isolate components and ensure correctness at the code level.

Table 5.1: Unit Testing - Authentication Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
UT01	User Registration	Enter user information and submit registration form	Valid user details	User account created successfully	As Expected	Pass
UT02	User Login	Enter email and password then click login	Valid credentials	User redirected to dashboard	As Expected	Pass
UT03	Invalid Login	Enter incorrect password and submit	Invalid password	Error message displayed	As Expected	Pass
UT04	Password Reset	Request password reset link	Registered email	Reset link sent successfully	As Expected	Pass

Table 5.2: Unit Testing - Gig Management Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
UT05	Create Gig	Fill gig form and submit	Valid gig details	Gig created successfully	As Expected	Pass
UT06	Edit Gig	Update gig information	Updated gig data	Gig information updated	As Expected	Pass
UT07	Delete Gig	Delete existing gig	Existing gig	Gig removed successfully	As Expected	Pass
UT08	Search Gig	Search using keyword	Gig keyword	Relevant gigs displayed	As Expected	Pass

Table 5.3: Unit Testing - Payment Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
UT09	Payment Initiation	Proceed to checkout	Valid order	Payment request generated	As Expected	Pass
UT10	Payment Verification	Complete payment	Valid transaction	Payment verified	As Expected	Pass
UT11	Escrow Hold	Submit successful payment	Paid order	Funds held in escrow	As Expected	Pass
UT12	Escrow Release	Approve completed work	Completed order	Funds released to freelancer	As Expected	Pass

Table 5.4: Unit Testing - Messaging Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
UT13	Send Message	Send chat message	Valid message	Message stored successfully	As Expected	Pass
UT14	Receive Message	Open chat window	Existing conversation	Message displayed correctly	As Expected	Pass
UT15	Real-time Delivery	Send message between users	Two active users	Instant message delivery	As Expected	Pass
UT16	Chat History	Load previous messages	Existing conversation	Chat history retrieved	As Expected	Pass

5.2 Integration Testing

Integration testing was performed after unit testing to verify that different modules interact correctly with each other. The testing focused on communication between the Gig Service, Order Service, Payment Service, Messaging Service, and Authentication Service. Both bottom-up and top-down integration approaches were considered to ensure smooth data flow across the system.

Table 5.5: Integration Testing - Authentication and Profile Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
IT01	Registration Integration	Register a new account	Valid user details	User profile created automatically	As Expected	Pass
IT02	Login Integration	Login and access profile	Registered user	Profile loaded correctly	As Expected	Pass

Table 5.6: Integration Testing - Gig and Order Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
IT03	Gig Purchase	Select gig and place order	Existing gig	Order created successfully	As Expected	Pass
IT04	Order Tracking	Update order status	Existing order	Status reflected correctly	As Expected	Pass

Table 5.7: Integration Testing - Order and Payment Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
IT05	Payment Processing	Complete payment for order	Valid order	Order status updated to Paid	As Expected	Pass
IT06	Escrow Workflow	Complete order and approve work	Completed order	Funds released successfully	As Expected	Pass

Table 5.8: Integration Testing - Messaging and Notification Module

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
IT07	New Message Alert	Send message	Valid users	Notification generated	As Expected	Pass
IT08	Read Notification	Open notification	Existing notification	Status updated as read	As Expected	Pass

5.3 System Testing

System testing was performed on the fully integrated GigHub platform to verify that all functional and non-functional requirements were satisfied. The complete system was tested under realistic usage scenarios including authentication, gig management, messaging, payment processing, and review submission. This testing ensured that the platform met quality standards before deployment.

Table 5.9: System Testing - User Onboarding Workflow

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
ST01	Complete Registration	Register, verify email and login	New user	User account activated successfully	As Expected	Pass
ST02	Profile Setup	Complete profile information	Registered user	Profile updated successfully	As Expected	Pass

Table 5.10: System Testing - Freelance Marketplace Workflow

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
ST03	Gig Creation Workflow	Create and publish gig	Freelancer account	Gig visible in marketplace	As Expected	Pass
ST04	Gig Purchase Workflow	Purchase gig and place order	Client account	Order created successfully	As Expected	Pass

Table 5.11: System Testing - Payment and Escrow Workflow

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
ST05	Payment Workflow	Complete checkout process	Valid order	Payment completed successfully	As Expected	Pass
ST06	Escrow Release Workflow	Approve completed work	Completed order	Funds released to freelancer	As Expected	Pass

Table 5.12: System Testing - End-to-End Workflow

Test ID	Test Title	Testing Steps	Test Data	Expected Result	Actual Result	Pass/Fail
ST07	Complete Marketplace Cycle	Register, create gig, purchase, deliver work, review	Client and Freelancer accounts	Entire workflow completed successfully	As Expected	Pass
ST08	Real-time Communication Workflow	Exchange messages during order process	Active users	Messages delivered instantly	As Expected	Pass

Chapter 6

Results

6.1 Expected Results

The GigHub system is expected to deliver a fully functional, scalable, and user-friendly mobile-based service marketplace tailored for university students. The anticipated outcomes are based on the system design, feasibility analysis, and implemented architectural model.

6.1.1 Functional Outputs

- The system will allow students to register, authenticate, and manage personal profiles securely.
- Users will be able to post, browse, and apply for various gig-based services.
- A structured marketplace will enable peer-to-peer service exchange within the university environment.
- The system will support real-time notifications for gig updates and transactions.
- An admin panel will manage users, content moderation, and system monitoring.

6.1.2 Performance Expectations

- The mobile application is expected to provide response times under 2–3 seconds for standard operations.
- The system is designed to handle up to approximately 10,000 active users with stable performance.
- Efficient database queries and caching mechanisms will reduce server load and latency.

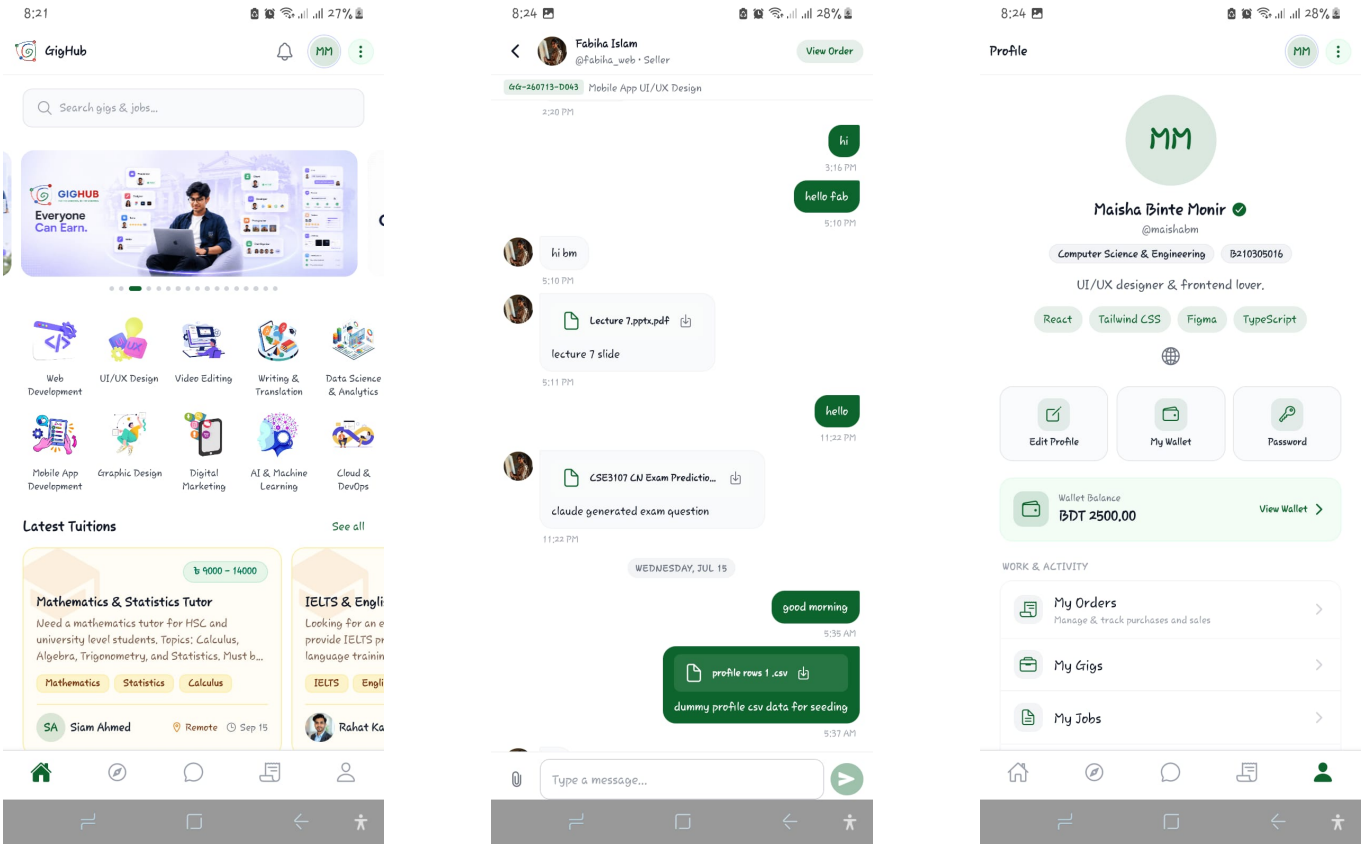
6.1.3 Usability and User Experience

- The React Native-based interface is expected to provide a smooth and consistent user experience across Android and iOS platforms.
- The UI will be simple, intuitive, and optimized for student usage patterns.
- Minimal training will be required for end-users due to a clean and familiar interface design.

6.2 System Output

- GigHub is expected to function as a scalable university-focused gig marketplace platform.
- The system will successfully bridge the gap between student skill supply and demand within the campus ecosystem.
- It is expected to encourage entrepreneurship, collaboration, and skill monetization among students.

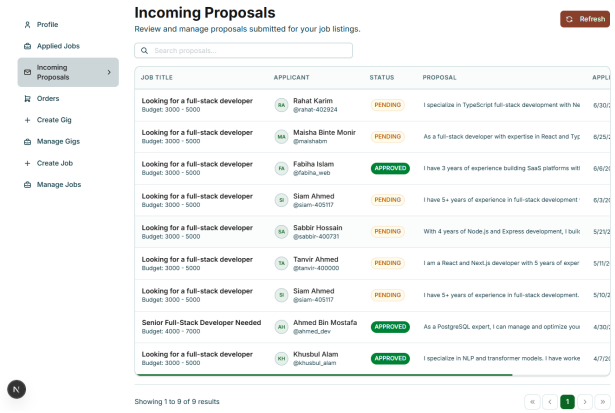
Summary: The proposed system is expected to be technically feasible, economically viable, and operationally effective within the defined constraints of the project.



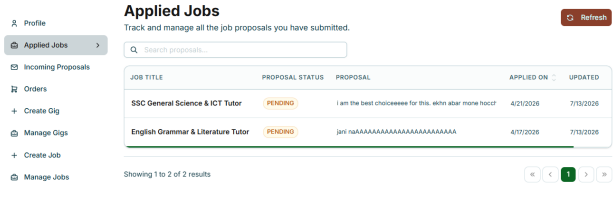
(a) Homepage (b) Chat with Clients (c) User Dashboard

Figure 6.1: App Screenshots

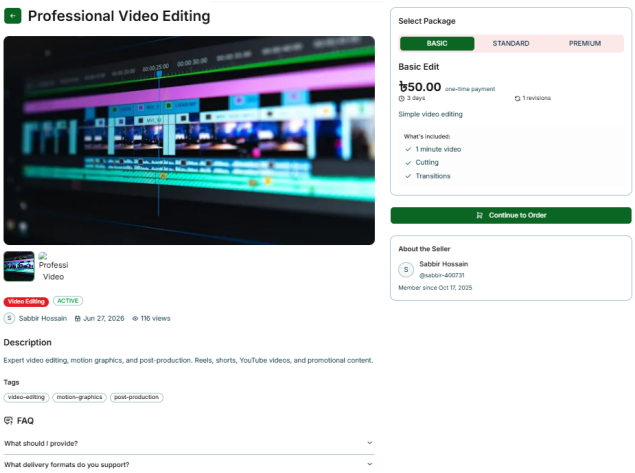
(a) Admin Dashboard



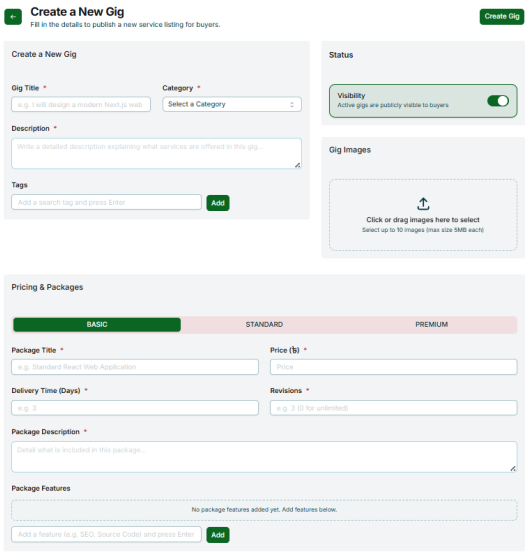
(b) Proposals



(c) Applied Jobs



(d) Job Description



(e) Gig Creation

Figure 6.2: System screenshots

Chapter 7

Conclusion

7.1 Summary

The GigHub project presents a mobile-based gig marketplace system developed using React Native, aimed at facilitating peer-to-peer service exchange among university students. The system integrates core functionalities such as user authentication, service posting, browsing, application management, and transaction handling within a unified platform.

From an engineering perspective, the project demonstrates a complete software development lifecycle including requirement analysis, system design, implementation planning, cost estimation, risk analysis, and feasibility evaluation. Economic analysis using both LOC-based and Function Point methods indicates that the system is feasible within a moderate budget and a 6-month development timeline. Risk assessment further confirms that both technical and schedule risks are manageable with appropriate mitigation strategies.

7.2 Future Work

The GigHub platform can be significantly enhanced in future iterations through the following developments:

- Expansion of the platform to multiple universities and regional institutions.
- Integration of secure digital payment gateways for automated transactions.
- Implementation of AI-based recommendation systems to match users with relevant gigs.
- Development of advanced reputation and review systems to improve trust and reliability.
- Introduction of real-time chat and communication features between users.
- Enhancement of scalability using microservices architecture for large-scale deployment.
- Incorporation of fraud detection and verification mechanisms to improve platform security.